In questo capitolo iniziamo ad investigare la potenza degli algoritmi nella risoluzione di problemi. Dimostreremo che alcuni problemi possono essere risolti in maniera algoritmica e altri no. Il nostro obiettivo è esplorare i limiti della risolvibilità algoritmica dei problemi. Perché dovremmo studiare anche l'insolubilità? Serve studiare questo fenomeno per due motivi:

1. Ci rendiamo conto che il problema deve essere semplificato o modificato prima che possiamo trovare una soluzione algoritmica;
2. la seconda ragione è di tipo culturale.

4.1 LINGUAGGI DECIDIBILI

In questa sezione diamo alcuni esempi di linguaggi che sono decidibili mediante algoritmi. Ci concentriamo su linguaggi che riguardano degli automi e delle grammatiche.

PROBLEMI DECIDIBILI RELATIVI A LINGUAGGI REGOLARI

Forniamo algoritmi per testare se un automa finito accetta o meno una stringa, se il linguaggio di un automa finito è vuoto e se due automi finiti sono equivalenti.

Sia dato il problema di verificare se un automa a stati finiti deterministico accetta una stringa. Più formalmente:
$A_{DFA} = \{\langle B, w\rangle \mid B \text{ è un DFA che accetta la stringa in ingresso } w\}$.
Si dimostri il seguente problema:

Teorema 4.1
A_{DFA} è un linguaggio decidibile

IDEA: Dobbiamo trovare una MdT deterministica che lo risolva.

M = "su ingresso $\langle B, w\rangle$:
1. simula B su w;
2. se la simulazione finisce in uno stato finale, allora accetta, altrimenti rifiuta."

DIM: B non è altro che la quintupla che definisce un DFA, e quindi $\{Q, \Sigma, \delta, q_0, F\}$. Quindi come prima cosa M verificherà che B effettivamente sia un DFA (altrimenti rifiuta subito), quindi simulerà la stringa w direttamente su di lui. Come? Scrivendo ad ogni passo sul nastro lo stato e la posizione corrente di B sull'ingresso w, che sarà inizialmente q_0 e dovrà rispettare ad ogni step la funzione di transizione δ. Quando M arriverà a processare l'ultimo simbolo di w, dovrà verificare se è arrivato in uno degli stati finali accettanti del DFA, e accettare o rifiutare la stringa di ingresso di conseguenza.

Possiamo dimostrare un teorema simile per automi a stati finiti non deterministici. Dato:
$A_{NFA} = \{\langle B, w\rangle \mid B \text{ è un NFA che accetta la stringa di input } w\}$.

Teorema 4.2
A_{NFA} è un linguaggio decidibile

DIM: Quello che cambia rispetto al teorema 4.1 è che ora abbiamo a che fare con un automa non deterministico. Poco male, sapendo che ogni NFA ha un DFA corrispondente, definiamo la MdT non deterministica che lo risolve:

N = "su ingresso $\langle B, w\rangle$:
1. converti NFA B in DFA C;
2. Esegui la MdT M del teorema 4.1 sull'ingresso $\langle C, w\rangle$;
3. Se M accetta, allora accetta; altrimenti rifiuta."

In modo analogo, possiamo determinare se un'espressione regolare genera una data stringa. Consideriamo:
$A_{REX} = \{\langle R, w\rangle \mid R \text{ è un'espressione regolare che genera la stringa } w\}$

/ /

Teorema 4.3

A_{REX} è un linguaggio decidibile.

DIM: Definiamo la MdT P che decide A_{REX}:

P = "Su input $\langle R, w \rangle$:

1. Converte l'espressione regolare R in un NFA A equivalente, mediante la procedura di conversione data nel Teorema 1.54;
2. Esegue la MdT N del teorema 4.2 su input $\langle A, w \rangle$;
3. Se N accetta, accetta; se N rifiuta, rifiuta."

Ora affrontiamo un diverso tipo di problema concernente gli automi a stati finiti: il test del vuoto per il linguaggio di un automa finito. Nei precedenti teoremi dovevamo decidere se un automa finito accettasse una particolare stringa. Nella prossima dimostrazione, dobbiamo determinare se un automa finito accetta una qualche stringa. Consideriamo:

$E_{DFA} = \{\langle A \rangle / A \text{ è un DFA e } L(A) = \emptyset\}$

Teorema 4.4

E_{DFA} è un linguaggio decidibile

DIM: Verificare che un DFA non accetti alcuna stringa significa dimostrare che il suo linguaggio è vuoto. Viceversa, si ricordi che un DFA accetta una stringa se e solo se è possibile raggiungere uno stato finale utilizzando le sue funzioni di transizione (quindi percorrendo il verso delle frecce del DFA). Diventa dunque facile definire una MdT deterministica T che risolva E_{DFA}:

T = "Su ingresso $\langle A \rangle$:

1. Marca lo stato iniziale di A;
2. Ripeti finché non trovi nuovi nuovi stati marcati:
   3. Marca un nuovo stato con transizione proveniente da uno stato già marcato;
4. Se lo stato finale è marcato, rifiuta; altrimenti accetta."

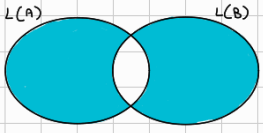
Il prossimo teorema afferma che determinare se due DFA riconoscono lo stesso linguaggio è decidibile. Sia:

$EQ_{DFA} = \{\langle A, B \rangle / A \text{ e } B \text{ sono DFA e } L(A) = L(B)\}$.

Teorema 4.5

EQ_{DFA} è un linguaggio decidibile.

DIM: Per la dimostrazione si procederà a definire un terzo automa a stati finiti C che accetti le stringhe di A o quelle di B, ma non entrambe. Per realizzarlo si potranno usare solo le operazioni di intersezione, unione o complemento, perché sono chiuse rispetto ai linguaggi regolari. In particolare si utilizzerà la formula della differenza simmetrica:



$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

In particolare: $L(C) = \emptyset$ se e solo se $L(A) = L(B)$. Il problema EQ_{DFA} su A e B può essere dunque ricondotto a un problema EQ_{DFA} su C. Costruiamo dunque la MdT deterministica F che risolve EQ_{DFA}:

F = "su ingresso $\langle A, B \rangle$:

1. Costruisci C;
2. Esegui la MdT T del teorema 4.4 su $\langle C \rangle$;
3. Se T accetta, allora accetta; altrimenti rifiuta."

PROBLEMI DECIDIBILI RELATIVI A LINGUAGGI CONTEXT-FREE

Descriviamo ora algoritmi per determinare se una CFG genera una particolare stringa e per determinare se il linguaggio di una CFG è vuoto. Consideriamo: $A_{CFG} = \{\langle G, w\rangle \mid G \text{ è una CFG che genera la stringa } w\}$.

Teorema 4.7

A_{CFG} è un linguaggio decidibile.

DIM: Metterci a fare tutte le derivazioni di G per vedere se una di queste è una derivazione di w sarebbe folle, perché nel caso in cui non fosse vero potremmo andare avanti all'infinito. L'alternativa è usare la grammatica G in forma normale di Chomsky, perché in questo caso le derivazioni di w sarebbero solo $2n-1$ (dove n è la lunghezza della stringa), e avremmo la garanzia di trovare una soluzione in un numero finito di passi. Costruiamo allora la MdT S che risolve A_{CFG}:

S = "su ingresso $\langle G, w\rangle$:
1. Converti G nella forma equivalente di Chomsky;
2. Fai una lista di tutte le derivazioni di $2n-1$ passi, dove $n = |w|$;
3. Se una derivazione genera w, allora accetta; altrimenti rifiuta."

Torniamo ora al problema del test del vuoto per il linguaggio di una CFG. Come abbiamo fatto per i DFA, possiamo dimostrare che il problema di determinare se una CFG genera almeno una stringa è decidibile. Sia:
$E_{CFG} = \{\langle G\rangle \mid G \text{ è una CFG e } L(G) = \emptyset\}$

Teorema 4.8

E_{CFG} è un linguaggio decidibile.

DIM: Per verificare se la grammatica è vuota dobbiamo dimostrare di non riuscire ad arrivare ad una stringa composta solo da terminali. Come prima cosa dovremo scorrere la CFG a marcare tutti i terminali. Successivamente si passa a scansionare le regole, e se da una di queste risulta che una variabile può essere sostituita da terminali marcati, marchiamo anche lei. L'operazione va ripetuta marcando tutte le variabili ottenute a partire da simboli (terminali/variabili) già marcati. Se si arriva a marcare anche la variabile iniziale significa che da lei posso raggiungere una sequenza composta solo da terminali, quindi la MdT R che risolve E_{CFG} dovrà rifiutare tale eventualità.

R = "su ingresso $\langle G\rangle$:
1. Marca i simboli terminali di G;
2. Ripeti finché non ci sono altre variabili marcate:
   3. Marca ogni variabile A in cui G abbia una regola $A \to U_1 \dots U_k$ in cui ogni $U_1 \dots U_k$ sia già marcato;
4. Se la variabile iniziale non è marcata, allora accetta; altrimenti rifiuta."

Consideriamo ora il problema di determinare se due grammatiche context-free generano lo stesso linguaggio. Sia:
$EQ_{CFG} = \{\langle G, H\rangle \mid G \text{ e } H \text{ sono CFG e } L(G) = L(H)\}$

Abbiamo utilizzato la procedura di divisione per E_{DFA} per dimostrare che EQ_{DFA} è decidibile. Poiché E_{CFG} è anch'esso decidibile, si potrebbe pensare che possiamo usare una strategia simile per dimostrare che EQ_{CFG} è decidibile. Ma qualcosa non va con questa idea: la classe dei CFL non è chiusa per complemento o intersezione. Infatti EQ_{CFG} non è decidibile. Una dimostrazione di ciò viene presentata nel capitolo 5. Ora dimostriamo che i CFL possono essere decisi dalle MdT.

Teorema 4.9

Ogni linguaggio context-free è decidibile.

IDEA: Sia A una CFL. Il nostro obiettivo è mostrare che A è decidibile. Una (cattiva) idea è quella di convertire un PDA per A direttamente in una MdT. Ciò non è difficile da realizzare, perché simulare una pila con il nastro di una MdT è semplice. Il PDA per A può essere non deterministico, ma ciò sembra andare bene, perché siamo in grado di convertirlo in una MdT non deterministica e sappiamo che ogni MdT non deterministica può essere convertita in una MdT deterministica equivalente. C'è tuttavia una difficoltà. Alcuni rami della computazione del PDA possono andare avanti per sempre. La MdT che effettua la simulazione avrebbe quindi alcuni cammini che non terminano mai, quindi la MdT non sarebbe un decisore.

DIM: E' necessaria una diversa soluzione. Sia A un qualsiasi CFL generato dalla CFG G. Per la dimostrazione sfruttiamo la MdT S che abbiamo progettato nel teorema 4.7 per decidere A_{CFG}, e scriviamo la MdT M_G che decide A:

M_G = "su ingresso w:
1. Esegui S su ingresso $\langle G, w \rangle$;
2. Se S accetta, allora accetta; altrimenti rifiuta."

4.2 INDECIDIBILITÀ

In questa sezione e nel capitolo 5, incontrerete vari problemi computazionalmente non risolvibili. L'obiettivo è quello di sviluppare la capacità di riconoscere un problema irrisolvibile e di imparare alcune tecniche per dimostrarne l'insolubilità.

Ora affronteremo il nostro primo teorema che stabilisce l'indecidibilità di uno specifico linguaggio: il problema di determinare se una MdT accetta una determinata stringa in input. Chiamiamo tale linguaggio A_{TM} per analogia con A_{DFA} e A_{CFG}. Tuttavia, mentre A_{DFA} e A_{CFG} sono decidibili, A_{TM} non lo è. Sia: $A_{TM} = \{\langle M, w \rangle \mid M \text{ è una MdT e } M \text{ accetta } w\}$.

Teorema 4.11
A_{TM} è indecidibile.

Osserviamo che A_{TM} è Turing-riconoscibile. Quindi questo teorema dimostra che i riconoscitori sono più potenti dei decisori. Scrivere una MdT U che riconosce A_{TM} è decisamente semplice:

U = "su ingresso $\langle M, w \rangle$:
1. Esegui M sull'ingresso w;
2. Se M entra nello stato di accettazione, accetta; se M entra invece nello stato di rifiuto, rifiuta."

Qual è il problema allora? Che se M non termina su w, allora la macchina va in loop sull'ingresso $\langle M, w \rangle$. Il fatto che non possa uscire da questo loop rende il problema indecidibile. La macchina di Turing U è di per sé, interessante. E' un esempio della prima macchina universale di Turing introdotta da Alan Turing nel 1936. Questa macchina è chiamata universale perché è in grado di simulare qualsiasi altra macchina di Turing a partire dalla descrizione di tale macchina.

L'indecidibilità la dimostriamo con il metodo della diagonalizzazione scoperto dal matematico Cantor per misurare la dimensione degli insiemi infiniti. Poiché in collezioni infinite non è possibile contare gli elementi che ne fanno parte, per capire se due insiemi sono della stessa dimensione si verifica se è possibile mettere in corrispondenza gli elementi dell'uno con quelli dell'altro. Più formalmente, due insiemi infiniti A e B sono della stessa dimensione se esiste una corrispondenza $f: A \longrightarrow B$. Per corrispondenza si intende una funzione biettiva da A a B, quindi ogni elemento di A può corrispondere a un solo elemento di B, e ogni elemento di B un unico elemento di A che corrisponde ad esso.

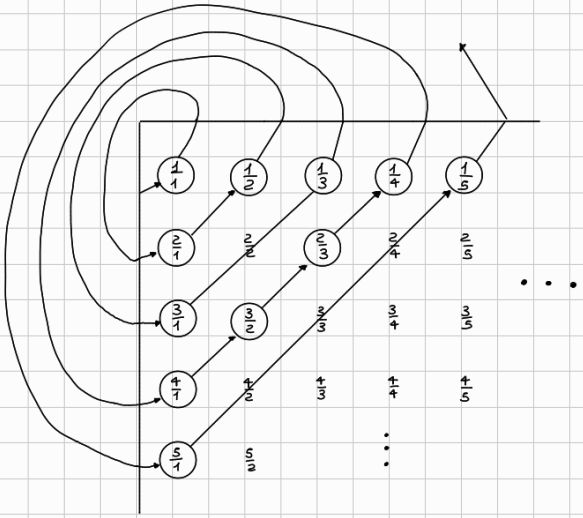
Facciamo un esempio: l'insieme N dei numeri naturali ha la stessa dimensione dell'insieme E dei numeri pari? Esiste una corrispondenza tra i due? Sì = $f(n) = 2n$. Quindi hanno la stessa dimensione.

Anche se sembrerà di starci allontanando dall'obiettivo (dimostrare che A_{TM} è indecidibile), dobbiamo introdurre un po' di teoremi e dimostrazioni di tipo matematico. Torneranno utilissime poi.

Definizione 4.14
Un insieme A è numerabile se è finito oppure ha la stessa cardinalità di N.

Per la dimostrazione usiamo come esempio quello dei numeri razionali positivi Q, così definibili: $Q = \{(m/n) \mid m, n \in N\}$

Trovare una corrispondenza tra i due insiemi non è banale, ma comunque possibile. Come prima cosa creiamo una matrice bidimensionale $|N| \times |N|$ e scriviamo su ogni cella (i, j) il numero razionale i/j. A questo punto creiamo una lista di numeri spostandoci sulle diagonali ed evitando le ripetizioni (ad esempio $\frac{1}{1}$ e $\frac{2}{2}$ sono lo stesso valore, quindi li contiamo una sola volta), secondo il seguente schema:



Questo schema dimostra l'esistenza di una corrispondenza tra $\mathbb{N}$ e $\mathbb{Q}$, quindi i due insiemi hanno la stessa dimensione.

Teorema 4.17

L'insieme dei numeri reali $\mathbb{R}$ è non numerabile

DIM: Dimostriamo il teorema per assurdo, sostenendo che $\mathbb{R}$ è numerabile e che quindi esiste una sua corrispondenza con $\mathbb{N}$. Questo ci semplifica la vita perché basterà trovare una $x \in \mathbb{R}$ che non sia compresa nella corrispondenza per mostrare la contraddizione della tesi.

Supponiamo che $f(n)$ sia una corrispondenza tra $\mathbb{R}$ e $\mathbb{N}$, e costruiamoci su misura una x che appartenga ad $\mathbb{R}$ ma non abbia corrispondenza in $\mathbb{N}$. Per capire partiamo da un esempio:

n	f(n)
1	3.14159...
2	55.5555...
3	0.12345...
4	0.50000...
⋮	⋮

Costruiamo la x in questo modo: $x = 0.c_1c_2...c_n$, dove la cifra decimale c_i deve essere diversa dalla i-sima cifra decimale dell'i-simo elemento. Riprendendo il nostro esempio:

n	f(n)
1	3.**1**4159...
2	55.**5**555...
3	0.12**3**45...
4	0.500**0**0...
⋮	⋮

$\Rightarrow \quad x = 0.4641...$

Questa costruzione ci dà garanzia che x sia diverso da qualsiasi elemento di $\mathbb{R}$ messo in corrispondenza con $\mathbb{N}$. Dato che x sfugge alla corrispondenza pur appartenendo all'insieme dei numeri reali, $\mathbb{R}$ e $\mathbb{N}$ non sono della stessa dimensione, e in particolare $\mathbb{R}$ non è numerabile. Si noti che in questa dimostrazione si è sfruttato ancora un metodo che sfrutta le diagonali.

Corollario 4.18

Alcuni linguaggi non sono Turing-riconoscibili.

DIM: la chiave della dimostrazione è che l'insieme dei linguaggi possibili non è numerabile, mentre quello delle MdT sì; dato che una MdT riconosce un solo linguaggio, alcuni linguaggi rimarranno tagliati fuori: quelli non Turing-riconoscibili. Dobbiamo verificare ogni affermazione.

1. L'insieme delle MdT è numerabile

   Dato un generico alfabeto Σ, l'insieme di tutte le stringhe Σ^* da lui definite è numerabile perché è possibile fare una lista di tutte le stringhe di una certa lunghezza. Dal momento che ogni MdT può essere codificata come una stringa $\langle M \rangle$, anche l'insieme delle MdT è numerabile.

2. L'insieme dei linguaggi non è contabile

   Definiamo sequenza binaria infinita una sequenza infinita di 0 e 1. Definiamo B l'insieme di tutte le sequenze binarie infinite, e si può dimostrare che non è numerabile utilizzando lo stesso metodo della diagonalizzazione usato per l'insieme dei numeri reali $\mathbb{R}$.
   Definiamo ora L l'insieme dei linguaggi definiti dall'alfabeto Σ e cerchiamo una relazione tra B e L. Lo scopo è chiaro: se troviamo tale relazione abbiamo dimostrato che l'insieme dei linguaggi non è contabile.

   Dato un linguaggio A appartenente ad L, definiamo sequenza caratteristica di A (χ_A) la sequenza binaria in cui ogni bit vale 1 se A usa una possibile stringa del linguaggio, 0 altrimenti. Facciamo un esempio:

   $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$
   $A = \{\quad 0, \quad 00, 01, \quad\quad 000, 001, \dots\}$
   $\chi_A = \{0\ 1\ 0\ 1\ 1\ 0\ 0\ 1\ \quad 1 \dots\}$

   Possiamo dunque affermare che esiste una funzione $f: L \to B$, dove $f(A)$ è la sequenza caratteristica di A che è unica in B. L'esistenza della funzione ci permette di sostenere che se B non è numerabile nemmeno l'insieme dei linguaggi L lo è.

Siamo ora pronti a dimostrare il Teorema 4.11, l'indecidibilità del linguaggio $A_{TM} = \{\langle M, w \rangle \mid M \text{ è una MdT e } M \text{ accetta } w\}$

DIM: Anche in questo caso la dimostrazione avverrà per assurdo.
Supponiamo di avere un decisore H del linguaggio tale che:

$$H(\langle M, w \rangle) = \begin{cases} \text{accetta} & \text{se } M \text{ accetta } w; \\ \text{rifiuta} & \text{se } M \text{ rifiuta } w; \end{cases}$$

Costruiamo ora una MdT D che usi H come sottoprocedura e faccia esattamente l'opposto:
D = "su ingresso $\langle M \rangle$:
1. Esegui H sull'ingresso $\langle M, \langle M \rangle \rangle$; (dove M è la stringa che descrive la MdT)
2. Se H accetta, allora rifiuta; altrimenti accetta."

Non lasciatevi confondere dall'idea di attivare una macchina sulla sua stessa descrizione. Ciò è simile a un programma che chiama se stesso in input, qualcosa che si fa a volte nella pratica. Ad esempio, un compilatore è un programma che traduce altri programmi. Un compilatore per il linguaggio Python può essere esso stesso scritto in Python, quindi eseguire tale programma su se stesso avrebbe senso.

Ricapitolando:

$$D(\langle M \rangle) = \begin{cases} \text{accetta} & \text{se } M \text{ non accetta } \langle M \rangle \\ \text{rifiuta} & \text{se } M \text{ accetta } \langle M \rangle. \end{cases}$$

Cosa succede quando eseguiamo D con la sua stessa descrizione $\langle D \rangle$ in input? In tal caso otteniamo:

$$D(\langle D \rangle) = \begin{cases} \text{accetta} & \text{se } D \text{ non accetta } \langle D \rangle \\ \text{rifiuta} & \text{se } D \text{ accetta } \langle D \rangle. \end{cases}$$

Indipendentemente da ciò che D fa, essa è costretta a fare il contrario, il che ovviamente è una contraddizione. Quindi né la MdT D né la MdT H possono esistere.

Dove si usa la diagonalizzazione per la dimostrazione del teorema 4.11?
Ri-rappresentiamo il tutto con una matrice: come indice di ogni riga mettiamo la MdT H_i, e come indice delle colonne mettiamo la stringa $\langle M_j \rangle$. Se una H_i accetta la stringa $\langle M_j \rangle$, scriveremo "accetta" nella cella (i,j), "rifiuta" altrimenti. Aggiungiamo poi un'ultima riga per il decisore D, e un'ultima colonna per la stringa $\langle D \rangle$ che lo descrive. La riga D è riempita scrivendo l'opposto delle celle che si trovano sulla diagonale della stessa colonna.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
H_1	accept	reject	accept	reject		accept	
H_2	accept	accept	accept	accept	...	accept	...
H_3	reject	reject	reject	reject		reject	
H_4	accept	accept	reject	reject		accept	
⋮		⋮			⋱		
D	reject	reject	accept	accept		?	
⋮		⋮					

La contraddizione si ottiene in corrispondenza del punto interrogativo, dove l'entrata dovrebbe essere l'opposto di se stessa.

UN LINGUAGGIO NON TURING-RICONOSCIBILE

Nella sezione precedente abbiamo dimostrato che un linguaggio - precisamente A_{TM} - è non decidibile. Ora mostriamo un linguaggio che non è neppure Turing-riconoscibile. Il teorema seguente mostra che, se un linguaggio e il suo complemento sono entrambi Turing-riconoscibili, il linguaggio è decidibile. Quindi per qualunque linguaggio non decidibile, o esso non è Turing-riconoscibile oppure il suo complemento non è Turing-riconoscibile. Ricordiamo che il complemento di un linguaggio è il linguaggio costituito da tutte le stringhe che non sono nel linguaggio.

Diciamo che un linguaggio è coTuring riconoscibile se esso è il complemento di un linguaggio Turing-riconoscibile.

Teorema 4.22
Un linguaggio è decidibile se e solo se è Turing-riconoscibile e coTuring-riconoscibile.

DIM: Per abbreviare, consideriamo un linguaggio A e chiamiamo:
- A_{DEC} = A decidibile;
- A_{TU} = A Turing-riconoscibile;
- A_{COTU} = A coTuring-riconoscibile;

Dimostriamo entrambi i sensi del "se e solo se":
1. Se A_{DEC} allora A_{TU} e A_{COTU}
   Se un linguaggio A è decidibile è per forza di cose anche Turing-riconoscibile, e dato che il complemento di un linguaggio decidibile è anche esso decidibile, A sarà anche coTuring riconoscibile.

2. Se A_{TU} e A_{COTU} allora A_{DEC}
   Introduciamo il riconoscitore M_1 per il linguaggio A, e il riconoscitore M_2 per il complemento del linguaggio A. Definiamo ora una MdT che sia decisore per A:

M: "su ingresso w:
1. Esegui M_1 e M_2 in parallelo sull'ingresso w; (l'esecuzione di due macchine in parallelo significa che M ha due nastri, uno per simulare M_1 e l'altro per simulare M_2)
2. Se M_1 accetta, accetta; se M_2 accetta, rifiuta)."

Ora dimostriamo che M è un decisore per A:
- Dato che ogni stringa w può essere in A o nel suo complemento, o M_1 o M_2 la accetteranno;
- Dato che M termina quando o M_1 o M_2 accettano, M terminerà sicuramente;

Per entrambi i motivi, M è un decisore di A, e quindi A è decidibile.

Corollario 4.23
$\overline{A_{TM}}$ non è Turing-riconoscibile

DIM: Sappiamo che A_{TM} è Turing-riconoscibile. Se anche $\overline{A_{TM}}$ fosse Turing-riconoscibile, A_{TM} sarebbe decidibile. Il teorema 4.11 ci dice che A_{TM} è non decidibile, quindi $\overline{A_{TM}}$ non può essere Turing-riconoscibile.